zkInfer: Resource-Aware Distributed Proof Serving for Zero-Knowledge Machine Learning

Anonymous Author(s)

Abstract

Zero-knowledge machine learning (zkML) allows a service to return an inference result with a cryptographic proof of correct execution, but proof generation is slow and memory intensive. Partitioning exposes independently provable jobs and can reduce per-job cost, yet concurrent requests must still share finite prover resources.

We characterize partitioned zkML execution and find substantial variation across jobs, strongly stage-dependent memory demand, and repeated circuit executions that support predictable resource profiles and reusable setup artifacts. We present *zkInfer*, a distributed runtime that separates cluster-wide job placement from node-local stage admission. The coordinator schedules jobs across requests and prover nodes, while each node admits stages using historical memory profiles and current local capacity. zkInfer also reuses circuit-specific setup artifacts without modifying the underlying proving protocol.

We evaluate zkInfer across convolutional, generative, and transformer workloads under single-node, distributed, and concurrent execution. **[Insert final concurrent-serving and scale-out headline results.]** These results show that scalable zkML serving depends not only on proving performance, but also on how proof work is scheduled across shared resources.

1 Introduction

Machine-learning inference is increasingly outsourced to remote services, as modern models often require more computation, memory, and specialized hardware than clients can practically provision locally [**zhang2024hardware**, **sheng2023flexgen**]. In this setting, the client observes the returned prediction but has no direct assurance that the specified model was evaluated correctly on the submitted input [**kang2024scaling**, **qu2025zkgpt**]. Re-executing the model locally would undermine the benefit of outsourcing, while inspecting the service's execution may require access to private inputs, model parameters, or intermediate values.

Zero-knowledge machine learning (zkML) addresses this problem by returning a cryptographic proof alongside the prediction [**chen2024zkml**, **kang2024scaling**, **boyuan2021zen**, **chen2025zktorch**]. The proof attests to correct execution without requiring the client to repeat the inference and can keep selected inputs or model values hidden. The resulting asymmetry is useful for verification but places most of the computational burden on the prover. zkML proof generation can be orders of magnitude slower than ordinary inference and consume large amounts of memory. For example, zkGPT [**qu2025zkgpt**] reports 147.7 s to prove the generation of 64 tokens compared with 94.55 ms for model execution, with peak prover memory reaching 240.5 GB.

Prior zkML work primarily reduces the cost of individual proofs through model transformations, optimized circuits, broader framework support, GPU acceleration, parallel proof accumulation, or partitioning [**boyuan2021zen**, **chen2024zkml**, **ezkl**, **sin2024zkllm**, **chen2025zktorch**, **sanjaya2026zkcomposer**]. These techniques improve proving efficiency and expose parallelism within an inference, but leave a complementary systems problem when multiple inference requests share finite prover resources. Each request may expand into many heterogeneous proof jobs whose runtime and memory demand vary substantially, while repeated executions can reuse expensive circuit-specific artifacts.

Our characterization in Section **??** shows that these properties make simple concurrency control inefficient. Launching too many jobs can oversubscribe memory, while reserving each job's peak demand for its full execution can leave capacity idle. A zkML serving runtime must therefore decide both where proof jobs execute and when resource-intensive stages may proceed.

Our Work. We present zkInfer, a distributed runtime for concurrent zkML proof serving. zkInfer transforms each inference request into independently executable proof jobs and uses a hierarchical resource-management architecture that separates cluster-wide placement from node-local admission. The coordinator balances requests, runtime estimates, node capacity, and setup locality when assigning jobs. Each prover node then admits execution stages using recent stage-level memory profiles and current local capacity, allowing safe overlap without reserving a job's peak memory for its entire lifetime. Reusable compiled circuits and proving keys are cached and reused across requests when available.

We implement zkInfer around existing zkML backends, using EZKL [**ezkl**] as the primary backend and Halo2 [**halo2**] as the underlying proof system. The proving protocol remains unchanged. We instrument the backend to expose execution stages and resource usage for characterization and admission decisions.

We evaluate zkInfer across convolutional, generative, and transformer workloads, including single-node, distributed, and concurrent execution. **[Add final concurrent-serving and scale-out headline results here once finalized.]**

Contributions. This paper makes three contributions:

- We characterize partitioned zkML inference as a serving workload, showing substantial variation across

SAC'27, Gwangju, South Korea
2027.

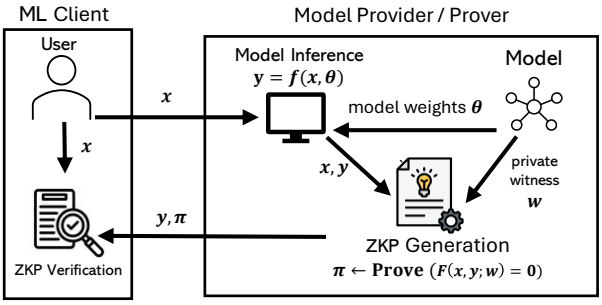


Figure 1: zkML inference and verification workflow. The model provider returns an output y and a proof π attesting that y was produced by evaluating the claimed model on input x.

proof jobs, stage-dependent memory demand, predictable repeated executions, and opportunities for setup reuse.

- We design and implement zkInfer, which separates cluster-wide job placement from node-local stage admission and incorporates historical resource profiles and artifact locality.
- We evaluate zkInfer across multiple model families and deployment scales, including concurrent request serving and distributed scale-out. [**Insert final headline results.**]

2 Background

2.1 zkML Workflow and Proving Pipeline

As shown in Figure **??**, a client submits an input x to a model provider, which evaluates a model with parameters θ to produce $y = f(x, \theta)$. The provider returns y together with a ZK-SNARK proof π attesting that the specified computation produced that output. Which values are public or private depends on the application.

To generate π, the model is mapped to a proof-compatible representation, often using quantized integer or fixed-point arithmetic [**boyuan2021zen**, **chen2024zkml**], and compiled into an arithmetic circuit or constraint system [**gabizon2019plonk**]. Circuit-specific proving and verification keys can be prepared ahead of time or generated on demand and reused across requests that share the same circuit configuration. For each request, the prover constructs the *witness*, containing the private values required to satisfy the circuit constraints, and generates π. The client verifies the proof using the verification key without re-executing the inference.

2.2 Proof Generation as a Systems Bottleneck

zkML proving is substantially more expensive than ordinary model execution because the prover must demonstrate that an arithmetic constraint system is satisfied. Operations such as fixed-point arithmetic, nonlinearities, comparisons, and range

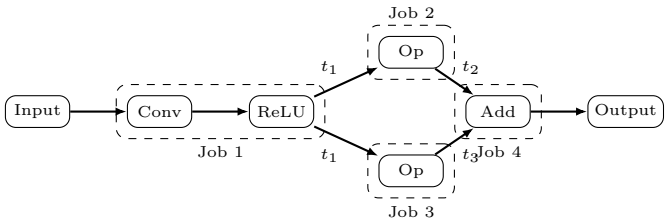


Figure 2: Example inference graph partitioned into four proof jobs. Intermediate tensors t_1, t_2, and t_3 cross job boundaries and define the interfaces between jobs.

checks may introduce additional constraints or lookup arguments [**chen2024zkml**, **kang2024scaling**, **liu2021zkcnn**]. Proof generation also materializes witness, circuit, proving-key, and polynomial data that can make the working set much larger than the model's inference activations.

These costs create systems-level constraints. Long proof times increase request latency, while large working sets restrict both the size of proof jobs that can execute on a machine and the number that can overlap safely. The problem becomes more pronounced in a serving setting, where multiple inference requests compete for finite compute and memory.

2.3 Model Computation as a Graph

Many zkML pipelines represent a model as an ONNX computation graph whose nodes are tensor operators and whose edges carry intermediate tensors [**onnx-paper**]. Figure **??** shows how this graph can be partitioned into proof jobs. Each job contains one or more operators and proves the corresponding portion of the computation. When an edge crosses a partition boundary, the upstream tensor becomes an input to the downstream job. Partitioning therefore determines both the computation assigned to each proof job and the intermediate values that connect jobs, providing a natural structure for independently schedulable proof work.

3 Characterizing zkML Serving Workloads

We characterize the properties of partitioned zkML workloads that shape scheduling in a shared prover. We focus on three questions: how partitioning changes proving cost, how runtime and memory vary across proof jobs, and how memory demand evolves within a job.

3.1 Workloads and Measurement Method

We use TinyBERT, MobileNetV2, and NanoGPT (4L), spanning convolutional, encoder, and autoregressive models. At the finest granularity they produce 120, 100, and 181 proof jobs, respectively. We compare monolithic proving with partitioned proving in which all jobs execute sequentially on the same prover, isolating the effect of proving granularity from inter-job parallelism. Both use EZKL [**ezkl**] on a dual-socket AMD EPYC 9354 server with 755 GiB of RAM. We record

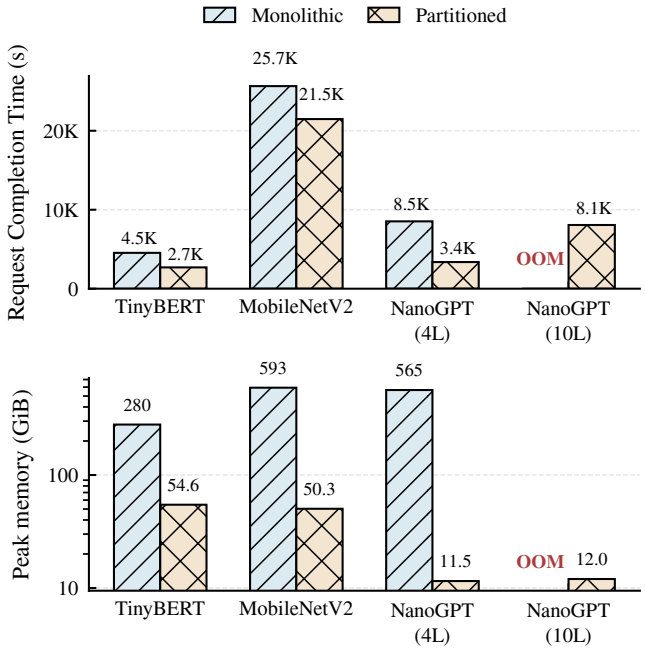


Figure 3: Monolithic and partitioned proving. Partitioned request time is the sum of proof-job runtimes; memory is the largest observed job footprint. OOM indicates that proving exceeded available memory.

per-job execution time, process memory, and major proof-stage transitions. Unless otherwise stated, runs begin cold so that the full setup and proving lifecycle is represented.

3.2 Impact of Partitioning on Proving Cost

Figure **??** shows that partitioning reduces proving time even when jobs execute serially. Relative to monolithic proving, completion time falls by 40.6% for TinyBERT, 16.3% for MobileNetV2, and 60.5% for NanoGPT (4L). Peak memory falls by 80.5%, 91.5%, and 98.0%, respectively. Partitioning also makes larger models feasible: monolithic NanoGPT (10L) exhausts available memory, whereas partitioned execution completes with a 12.0 GiB peak.

Implication. Partitioning changes the workload presented to the runtime by reducing the cost and memory footprint of individual proof jobs while exposing additional parallelism.

3.3 Heterogeneity Across Proof Jobs

Figure **??** shows substantial heterogeneity within each request. TinyBERT's longest job takes 1,461.3 s compared with a 4.47 s median, a 327× difference, and accounts for 40.9% of aggregate job time. MobileNetV2 and NanoGPT show smaller but still substantial spreads of approximately 21× and 31×.

Memory is similarly uneven. Median per-job peaks are 0.21, 3.38, and 0.22 GiB for TinyBERT, MobileNetV2, and NanoGPT, while the largest reach 51.61, 41.45, and 5.70 GiB.

Workload	Time at $\geq 90\%$ of peak (%)	Peak / average memory
TinyBERT	4.8	4.07×
MobileNetV2	3.5	2.53×
NanoGPT (4L)	13.3	2.22×

Table 1: Temporal variation in memory demand for the highest-memory job in each workload.

A fixed concurrency limit therefore says little about either remaining request work or instantaneous memory pressure.

Implication. Runtime estimates help identify jobs that may dominate request completion, while memory estimates determine which jobs can safely share a prover node.

3.4 Resource Demand Across Execution Stages

Peak memory does not reveal how demand evolves during execution. We therefore align process-memory samples with preparation, key generation, and proving. Figure **??** shows that the highest-memory jobs spend only 4.8%, 3.5%, and 13.3% of execution at or above 90% of peak memory for TinyBERT, MobileNetV2, and NanoGPT, respectively. Their peaks are 4.07×, 2.53×, and 2.22× their time-averaged memory consumption (Table **??**).

Implication. Whole-job peak reservation is conservative for much of execution. A runtime can expose additional concurrency by admitting stages according to their current memory demand rather than reserving every job's eventual peak for its entire lifetime.

4 zkInfer Design

4.1 System Overview

Building on Section **??**, we design zkInfer as a distributed runtime for serving zkML proofs under concurrent requests. A request consists of a model and inference input. zkInfer applies the configured partitioning policy, materializes the corresponding per-partition inputs, and creates proof jobs that can execute across a pool of prover nodes. The underlying proving protocol and proof-composition mechanism remain unchanged.

Figure **??** shows the architecture. The coordinator maintains jobs from all active requests and assigns whole jobs to prover nodes. Each node then controls when the assigned job's proving stages may run. This separation is central to the design: the coordinator handles request-level fairness, coarse placement, and setup locality, while the node supervisor uses current local memory and stage-level estimates for fine-grained admission.

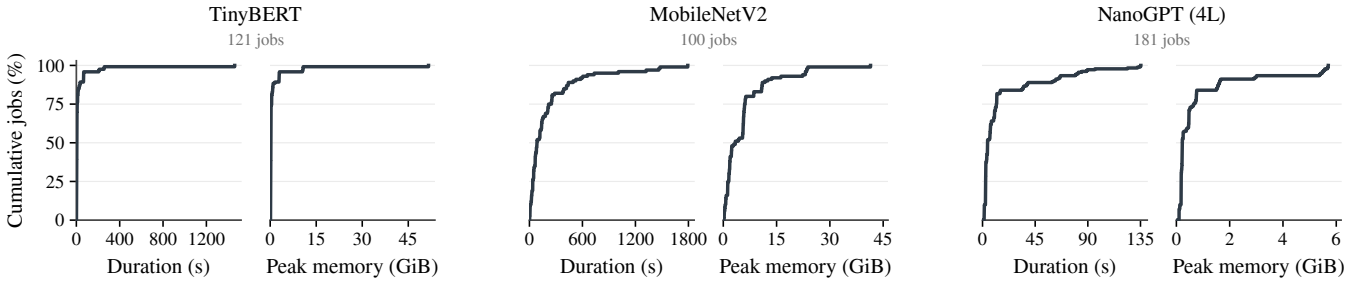


Figure 4: Empirical distributions of proof-job execution time and peak memory for TinyBERT, MobileNetV2, and NanoGPT (4L).

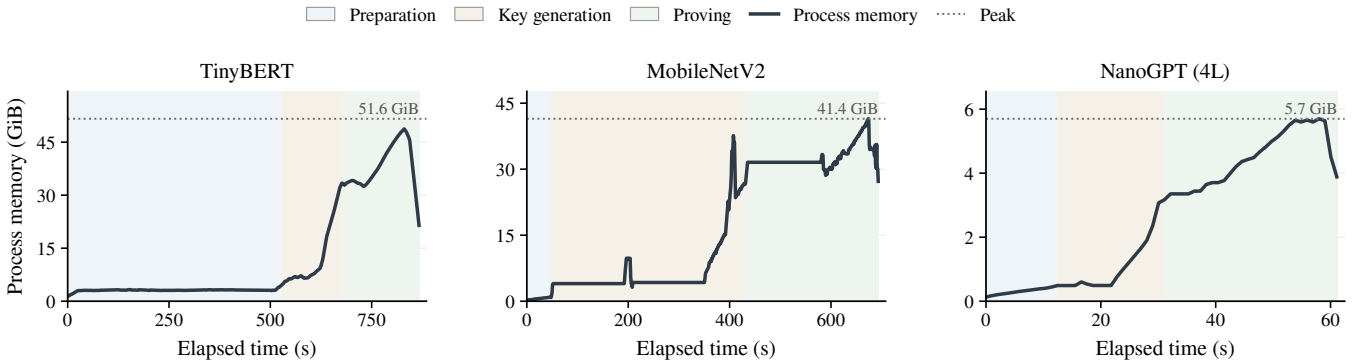


Figure 5: Memory demand over time for the highest-memory proof job in each workload. Shaded regions identify preparation, key generation, and proving.

4.2 Partitioning and Self-Contained Proof Jobs

During request preparation, zkInfer partitions the model graph according to a system-wide policy. Unless otherwise stated, we use one graph operator per proof job. Section **??** evaluates coarser partitions.

A partition may consume tensors produced elsewhere in the graph. zkInfer therefore executes the model once before distributed proving to materialize values that cross partition boundaries and attaches them to the consuming jobs as witness inputs. Each job can then be scheduled independently of the execution order of other jobs. Correctness does not rely on trusting these values directly. Each partition is proved separately, while the backend's existing proof-composition mechanism binds values produced by upstream partitions to those consumed downstream [**ezkl-splitting**].

4.3 Cluster-Wide Job Placement

The coordinator first chooses a request using round-robin fairness, then selects a job from that request. Requests with no eligible job for the polling node are skipped for that poll, and a request moves to the back of the queue after an assignment if work remains. This prevents a large request from monopolizing newly available prover capacity without requiring preemption.

Let $\widehat{m}_j^{\max}$ and $\widehat{d}_j$ denote the predicted maximum memory demand and duration of job j. A job is ineligible for a node if its known peak exceeds that node's physical memory. The coordinator does not require the whole-job peak to fit within currently free memory, because later memory-intensive stages may wait locally. Among eligible jobs of the selected request, it first prefers jobs whose reusable setup artifacts are already cached on the polling node, then selects the job with the largest predicted duration, using submission order to break ties.

Stage-level packing remains local to each node. Keeping short-lived memory conditions out of the coordinator avoids duplicating admission logic and lets the node make decisions using the freshest view of its own usage.

4.4 Node-Local Resource Admission

Once a proof job is assigned to a node, the node supervisor controls memory admission. Instead of reserving the job's predicted peak for its entire execution, the supervisor admits each controllable stage according to that stage's predicted demand and current local capacity.

Stage-level estimates. For each model partition and proving configuration, the supervisor records recent peak memory observations separately for each stage. With fewer than five observations, zkInfer uses the largest observed

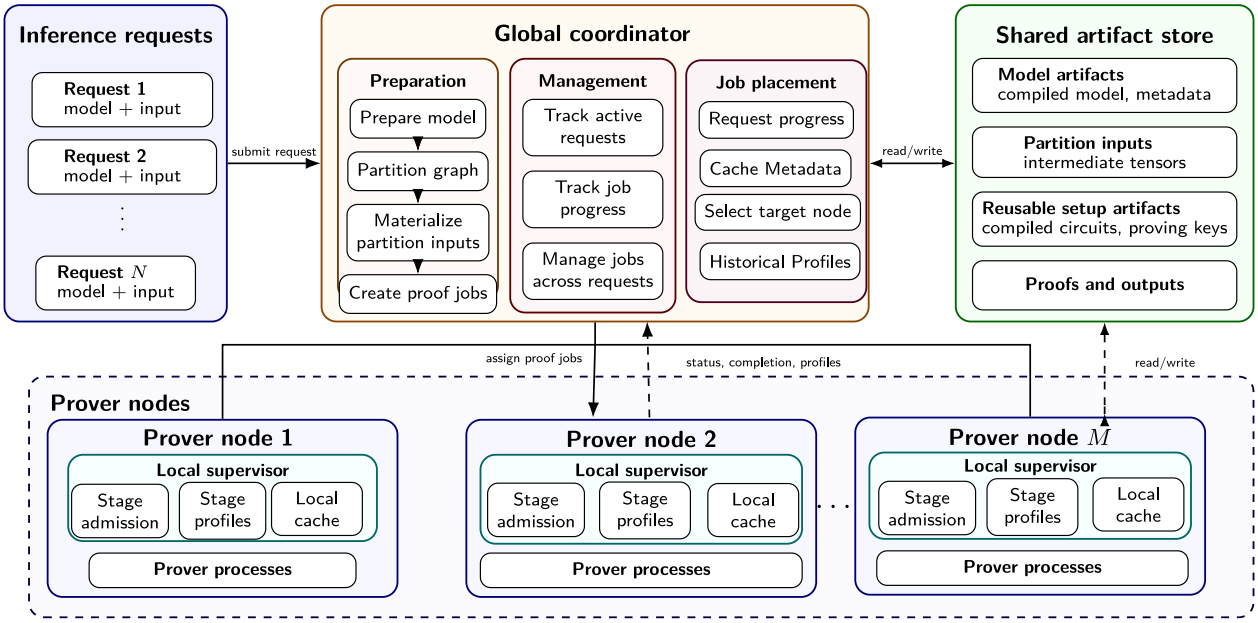


Figure 6: Architecture of zkInfer. A global coordinator places proof jobs across prover nodes, while each node performs local stage admission using current memory availability and historical resource profiles.

value. Once at least five are available, it uses the empirical 90th percentile of recent history. If no stage-specific history exists, the controller falls back to the job's whole-job estimate. Section **??** evaluates these predictions. Admission is memory-based because memory capacity determines whether multiple proof stages can execute safely on the same node.

Admission is enforced only at backend boundaries where execution can be delayed without interrupting an operation already in progress. EZKL already exposes major operations such as witness generation, setup, and proof generation. We additionally instrument setup to distinguish parameter loading, verification-key generation, and proving-key generation so that resource demand can be measured at finer granularity.

Reservation and live-memory checks. Let T_h be the physical memory of node h, R_h a fixed system reserve, and $B_h = T_h - R_h$ the budget available to proving work. Before stage s of job j begins, the supervisor obtains a predicted peak $\widehat{m}_{j,s}$. If A_h is the set of admitted stages, admission requires

$$\sum_{i \in A_h} \widehat{m}_i + \widehat{m}_{j,s} \le B_h. \tag{1}$$

The supervisor also checks live available memory to account for memory already resident in prover processes. Let r_i be the current resident memory of an admitted process and F_h the operating system's available memory after the reserve. The remaining protected growth is

$$U_h = \sum_{i \in A_h} \max(0, \widehat{m}_i - r_i). \tag{2}$$

For candidate stage (j, s), let $g_{j,s} = \max(0, \widehat{m}_{j,s} - r_j)$. Admission additionally requires

$$U_h + g_{j,s} \le F_h. \tag{3}$$

This avoids charging resident memory twice while preserving headroom for expected growth.

Once admitted, a stage runs to completion without preemption. Its reservation is released when the stage finishes, and the job must obtain a new reservation before entering the next controlled stage. If memory is insufficient, the job waits on the same node. A temporary startup reservation covers the interval between leasing a job and its first normal stage reservation.

4.5 Reusable Setup Artifacts

zkInfer persists reusable artifacts such as compiled circuits and proving keys after their first generation. Each node maintains a bounded local cache. A local hit avoids setup generation and restoration. If the artifact exists only in shared storage, the node restores it locally. Otherwise it is generated and persisted for later requests. During a cold execution, zkInfer keeps newly generated proving material in memory across setup and proving to avoid an unnecessary serialize-and-reload cycle.

Setup locality is a secondary placement objective. It is considered only after request selection and resource feasibility, so a cache hit does not override fairness or memory safety. When the cache exceeds its configured limit, inactive artifacts are evicted in recency order. Artifacts used by active jobs remain pinned until those jobs finish.

4.6 Leases, Failures, and Recovery

Proof jobs are also the unit of recovery. Each execution attempt receives a unique lease identifier. If a prover fails or a node stops responding, the coordinator returns that job to the pending queue while leaving completed jobs unchanged. Completion is accepted only when the reported lease matches the job's current lease, so late results from an earlier attempt are ignored. Heartbeats detect unavailable nodes, failed attempts are retried up to a configured limit, and optional coordinator persistence restores completed jobs while requeueing work that was running at restart.

5 Implementation and Deployment

We implemented zkInfer in approximately 12,000 lines of Python. The prototype includes the coordinator, node supervisors, isolated prover subprocesses, client API, and artifact storage. EZKL [**ezkl**] is the primary proving backend, with Halo2 providing the underlying proof system. We additionally modified the EZKL/Halo2 integration to expose proving stages and record the resource measurements used by zkInfer's admission policies and evaluation.

For colocated deployments, zkInfer stores artifacts on the local filesystem. Distributed deployments use Amazon S3 [**aws-s3**] as shared storage, with each prover node maintaining a bounded cache of reusable setup artifacts. The runtime interacts with proving backends through a common execution interface. Section **??** shows that this interface can also support zkTorch [**chen2025zktorch**].

6 Evaluation

We evaluate zkInfer by asking four questions: how effectively it serves concurrent requests, how stage-aware admission compares with whole-job peak reservation, whether historical stage profiles are accurate enough for online admission, and how much setup reuse reduces repeated work. We also measure single-request scale-out and sensitivity to partition granularity.

6.1 Experimental Setup

Platforms. Single-node experiments run on a dual-socket AMD EPYC 9354 server with 64 physical cores and 128 GiB of RAM. Multi-node experiments use up to 10 AWS m6a.4xlarge instances, each with 16 vCPUs and 64 GiB of RAM. One prover node also hosts the coordinator, and Amazon S3 is used for artifact transfer between the coordinator and prover nodes.

Workloads. We evaluate TinyBERT, MobileNetV2, and NanoGPT (4L), summarized in Table **??**. Unless otherwise stated, we use one ONNX operator per partition. Section **??** evaluates coarser partitions.

Model	Input	Params (M)	Proof jobs
TinyBERT	Seq. 96	14.5	120
MobileNetV2	$3 \times 224 \times 224$	3.5	100
NanoGPT (4L)	Seq. 64	4.2	181

Table 2: Primary evaluation workloads.

Policies. We compare three node-level execution policies while holding partitioning and coordinator scheduling fixed. *Fixed-k* limits each node to k active proof jobs. *Whole-job admission* reserves each job's predicted peak memory until completion. zkInfer instead reserves the predicted memory of the stage about to run. Unless otherwise stated, fixed concurrency uses $k = 8$.

Profiles and traffic. Cold runs generate setup artifacts during execution, while warm runs reuse artifacts generated beforehand. Historical profiles are collected in separate runs under the same setup condition and initialized identically across compared policies. Burst experiments submit $N \in \{1, 4, 8, 16\}$ requests simultaneously and report request latency, batch makespan, and peak memory.

6.2 Performance Under Concurrent Requests

Figure **??** compares the three policies as concurrent demand increases on a fixed prover node. At $N = 1$, the comparison isolates concurrency among jobs from one request. As N increases, jobs from multiple requests compete for the same machine. [**TO COMPLETE: Report the final latency and makespan differences, including the largest improvement over whole-job admission and where fixed concurrency remains competitive.**]

[**TO COMPLETE: Relate the observed differences to memory overlap, admission waiting, and CPU contention, and state the supported conclusion about stage-aware admission under increasing concurrent demand.**]

6.3 Single-Request Scale-Out

We next vary the prover pool from 1 to 10 nodes while holding the request and partitioning fixed. Let T_p denote completion time with p prover nodes and define speedup as T_1/T_p. Figure **??** reports completion time and speedup.

All three workloads benefit substantially from additional nodes. NanoGPT (4L) decreases from 1108 s to 89 s, MobileNetV2 from 9639 s to 762 s, and TinyBERT from 7900 s to 750 s, corresponding to 12.4×, 12.7×, and 10.5× speedups. At 10 nodes, summed job runtime per worker varies by only 116.8–118.2 s for NanoGPT and 557.1–560.1 s for MobileNetV2, indicating effective distribution of the same proof workload across workers.

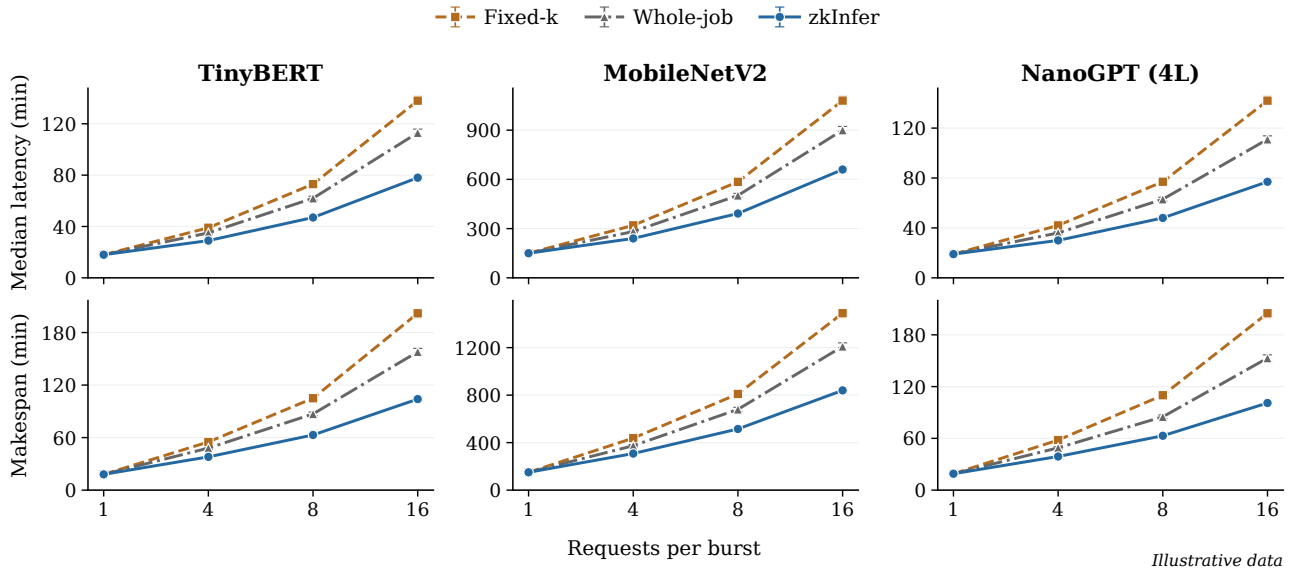


Figure 7: End-to-end performance under burst arrivals on a single prover node. We submit N requests simultaneously and compare fixed concurrency, whole-job admission, and zkInfer's stage-aware admission. Lower is better.

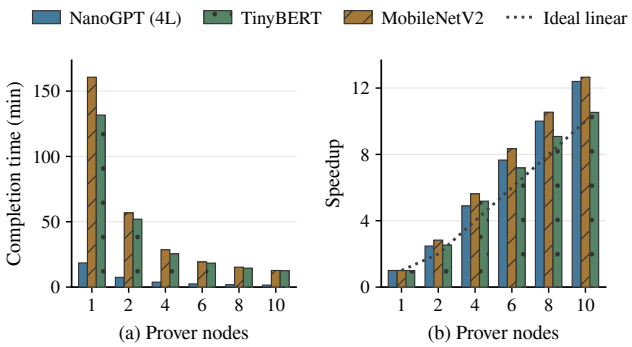


Figure 8: Single-request scale-out. (a) Request completion time. (b) Speedup relative to one node, with linear speedup shown dotted.

Execution breakdown. Table ?? shows that proof generation dominates compute time, while shared-storage cost is dominated by reads. Aggregate S3 read time remains broadly stable across the 1–10 node runs, ranging from 247–274 s for NanoGPT and 798–848 s for MobileNetV2. Increasing the prover pool therefore does not substantially increase transfer cost for a fixed request over the evaluated range.

6.4 Resource Profile Accuracy

zkInfer estimates stage memory from recent executions of the same model partition. We repeatedly execute representative partitions in isolation and compare each observed peak with the estimate available beforehand. With fewer than five prior measurements, zkInfer uses the largest observed peak.

	Compute		Shared storage	
Workload	Proving (s)	Verify (s)	S3 read (s)	S3 write (s)
MobileNetV2	4233.1	31.8	819.9	17.5
NanoGPT (4L)	746.3	7.1	264.0	20.3

Table 3: Mean aggregate job-level time across the scale-out runs. Times sum across jobs and may overlap in wall-clock execution.

Once five are available, it uses the empirical 90th percentile of recent history.

Figure ?? shows that median absolute prediction error remains below 0.5% in every measured configuration. Although 12.5–29.2% of predictions underestimate the subsequent peak, the largest observed shortfall is below 0.08 GiB. Thus, underprediction is usually a small deviation around an otherwise stable memory profile.

6.5 Setup Reuse and Partition Granularity

Setup reuse. Table ?? compares first-time generation with reuse from the node-local cache. Reuse removes 96.3–98.6% of aggregate setup time. The corresponding proving keys occupy 59.1–287.5 GiB across the three workloads, motivating a bounded cache rather than retaining every artifact locally.

Partition granularity. We vary the number of operators grouped into each proof job while holding the model and backend fixed. Figure ?? shows that coarser partitions increase aggregate prover time and peak job memory while exposing fewer independently schedulable jobs. We therefore use one operator per proof job in the main serving experiments.

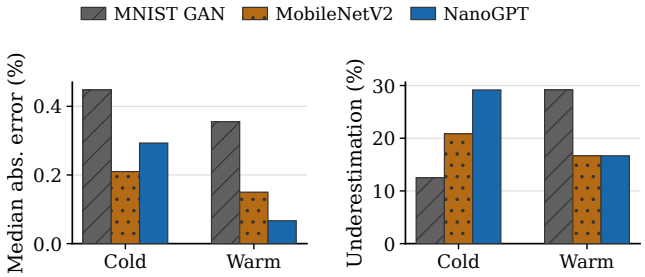


Figure 9: Stage-level memory prediction under cold and warm execution. The panels report median absolute error and underprediction frequency.

Workload	Gen. (s)	Reuse (s)	Saving (%)	PK (GiB)
TinyBERT	1,636.3	23.5	98.6	68.8
MobileNetV2	14,608.7	391.0	97.3	287.5
NanoGPT (4L)	1,812.3	67.1	96.3	59.1

Table 4: Cost of generating and reusing setup artifacts.

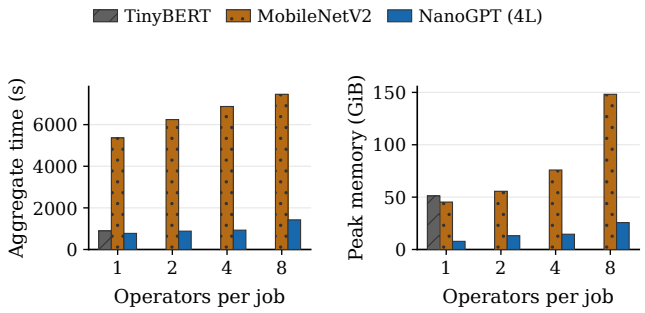


Figure 10: Effect of partition granularity on aggregate proving cost and peak job memory.

7 Related Work

zkML systems and prover optimization. Prior zkML systems reduce proving cost through specialized representations, protocols, and compilation techniques. ZEN [**boyuan2021zen**], Mystique [**weng2021mystique**], and zkCNN [**liu2021zkcnn**] optimize neural-network proving, while newer systems such as ZKML [**chen2024zkml**], zkLLM [**sin2024zkllm**], and ZKPyTorch [**xie2025zkpytorch**] extend support to broader workloads. zkInfer is complementary: it leaves the proving protocol unchanged and focuses on executing the resulting proof work under concurrent requests.

Partitioning and distributed proving. DIZK [**wu2018dizk**] and Pianist [**liu2024pianist**] distribute proof generation across machines to scale beyond a single prover. Within zkML, ZKTorch [**chen2025zktorch**] exposes parallelism through specialized basic blocks and parallel proof accumulation, while DSperse [**ivanov2025dsperse**] considers proofs over selected model slices. Recent orchestration work such as push0 [**ahmadvand2026push0**] addresses dispatch and recovery for distributed blockchain proving. These systems primarily scale an individual proof computation or proving pipeline. zkInfer instead treats proof jobs from concurrent inference requests as a shared serving workload with scheduling and resource management across the prover pool.

ML serving systems. Clipper [**crankshaw2017clipper**], Clockwork [**gujarati2020serving**], INFaaS [**romero2021infaas**], and Ray Serve [**rayserve**] manage concurrent inference workloads across shared resources. zkML serving differs in that one request may expand into long-running proof jobs with stage-varying memory demand and reusable proving artifacts. zkInfer applies serving principles to these proof-specific constraints through request-level placement and node-local admission.

8 Discussion

zkInfer uses memory as the primary admission resource because memory capacity determines whether proof stages can safely overlap on a node. The same framework could incorporate CPU estimates when CPU contention dominates. We also treat partitioning and runtime scheduling separately: partitioning defines the proof jobs, while zkInfer controls their execution. Finally, the coordinator provides request-level fairness rather than explicit deadlines or priorities, which could be added as alternative scheduling objectives.

9 Conclusion

zkInfer treats partitioned zkML proof generation as a distributed serving problem. It separates cluster-wide job placement from node-local stage admission, uses historical resource profiles to control memory safely, and reuses expensive setup artifacts across requests. Our evaluation shows that partitioning exposes heterogeneous proof jobs with strongly time-varying memory demand and that those jobs scale effectively across prover nodes. **[Insert final concurrent-serving headline result.]**